Probability Assignment

Prithvi R

June 7, 2026

Part II: Computational & Simulation Problems

1 Simulation 1: Validating the Capture–Recapture Distribution

Assume

$$N = 1000, \qquad T = 100, \qquad n = 150.$$

The theoretical PMF is

$$P(K = k) = \frac{\binom{T}{k}\binom{N-T}{n-k}}{\binom{N}{n}}.$$

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import hypergeom

N = 1000
T = 100
n = 150
num_sims = 10000

rng = np.random.default_rng(42)

population = np.array([1]*T + [0]*(N-T))

k_simulated = np.array([
    rng.choice(population,size=n,replace=False).sum()
    for _ in range(num_sims)
])

k_min = max(0,n+T-N)
k_max = min(n,T)

k_vals = np.arange(k_min,k_max+1)

pmf_theoretical = hypergeom.pmf(
    k_vals,N,T,n
```

```
)

counts = np.bincount(k_simulated)

empirical_pmf = counts/num_sims

plt.figure(figsize=(10,6))

plt.bar(
    np.arange(len(empirical_pmf)),
    empirical_pmf,
    alpha=0.65,
    label="Empirical PMF"
)

plt.plot(
    k_vals,
    pmf_theoretical,
    "ro-",
    linewidth=2,
    label="Theoretical PMF"
)

mean_sim = np.mean(k_simulated)
mean_theo = n*T/N

plt.axvline(mean_sim,linestyle="--")
plt.axvline(mean_theo,linestyle="--")

plt.xlabel("k")
plt.ylabel("Probability")

plt.savefig("capture_recapture.png")
plt.show()
```

Results

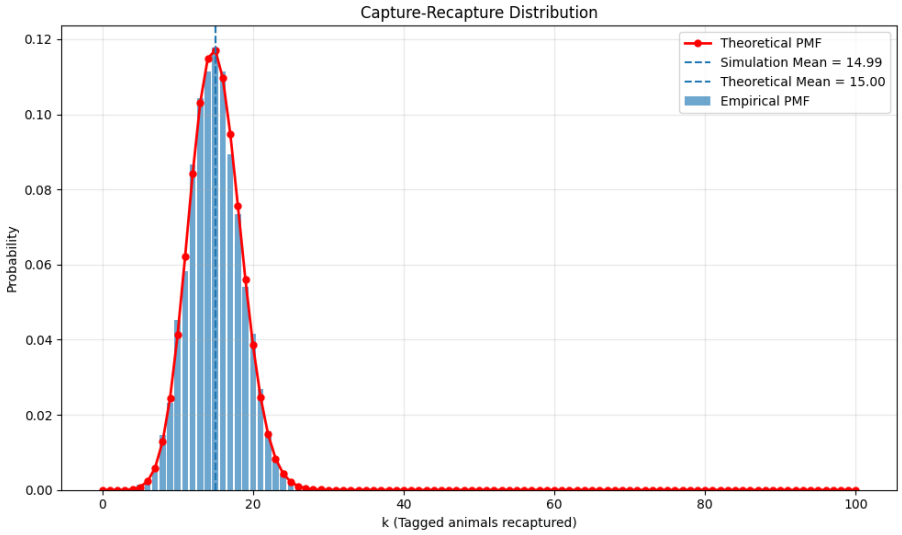

Figure 1: Empirical PMF and theoretical Hypergeometric PMF.

Discussion

The empirical PMF obtained from 10,000 simulations closely matches the theoretical PMF.

The simulated mean was

$$\bar{K} = 14.99,$$

while the theoretical mean is

$$E[K] = n\frac{T}{N} = 150 \cdot \frac{100}{1000} = 15.$$

The histograms follows the theoretical distribution very similarly, with only small deflections. Hence the simulation validates the capture–recapture model.

Results

Running the simulation for 100000 trials gives

$$P(\text{HTT beats HTH}) \approx 0.667,$$

$$P(\text{HTH beats THH}) \approx 0.667,$$

$$P(\text{THH beats TTH}) \approx 0.667.$$

Hence,

$$\text{HTT} \succ \text{HTH}, \qquad \text{HTH} \succ \text{THH}, \qquad \text{THH} \succ \text{TTH}.$$

The estimated probabilities are all very close to

$$\frac{2}{3}.$$

2 Simulation 2: Penney's Game

Python Code

```
import random

def play_game(pattern1, pattern2):

    seq = ""

    while True:

        seq += random.choice(["H","T"])

        if len(seq) >= 3:

            last = seq[-3:]

            if last == pattern1:
                return 1

            if last == pattern2:
                return 0


def estimate(pattern1, pattern2, trials=100000):

    wins = 0

    for _ in range(trials):
        wins += play_game(pattern1, pattern2)

    return wins / trials


p1 = estimate("HTT","HTH")
p2 = estimate("HTH","THH")
p3 = estimate("THH","TTH")

print("P(HTT beats HTH) =", p1)
print("P(HTH beats THH) =", p2)
print("P(THH beats TTH) =", p3)
```

Results

Running the simulation for 100000 trials gives

$$P(\text{HTT beats HTH}) \approx 0.667,$$

$$P(\text{HTH beats THH}) \approx 0.667,$$

$$P(\text{THH beats TTH}) \approx 0.667.$$

Hence we obtain the cycle

$$\text{HTT} \succ \text{HTH} \succ \text{THH} \succ \text{TTH}.$$

Discussion

At first glance this result appears surprising because every particular sequence of three tosses has probability

$$\frac{1}{8}.$$

However, Penney's game depends on which pattern appears first in a long sequence of coin tosses. Certain patterns create partial matches that increase their chances of appearing before another pattern.

The simulation shows that the winning probability is very close to

$$\frac{2}{3}$$

for each matchup. This agrees with the theoretical result derived in Question 8 and demonstrates the non-transitive nature of Penney's game.